UART : qui fait quoi ?

ATmega328P → port série du PC • Un guide pour résoudre l’exercice toi-même

Ton objectif

Comprendre comment configurer **115200 8N1**, puis demander l’envoi de **« Z » une fois par seconde**. Les exemples du guide utilisent d’autres valeurs ; les choix finaux restent à remplir.

1 RÉGLER

UBRR0H / UBRR0L
Choisir le diviseur d’horloge.

UCSR0A / B / C
Choisir mode, format et activation.

2 OBSERVER

UCSR0A
Lire les indicateurs d’état.

Le matériel dit si le tampon peut recevoir un nouveau caractère.

3 CONFIER

UDR0
Déposer un octet.

L’USART fabrique la trame et envoie les bits sur **PD1 / TXD**.

Le chemin d’un caractère



Une masse GND commune relie le MCU et l’adaptateur.

Le lien avec ton guide GPIO

PD1 est une broche du port D, mais elle possède une **fonction alternative TXD**. Quand l’émetteur USART est activé, il impose la sortie sur PD1, indépendamment de DDD1. L’USART pilote alors les bits.

UART / USART : l’UART est asynchrone. L’USART peut aussi être synchrone. Ici, tu utilises **USART0 en mode asynchrone**, sans ligne d’horloge XCK.

Sources : datasheet, §19.2 p.143-144 ; §19.5 p.148 ; §13.3.3 p.70-71

Registres, bits et masques

Un registre est un groupe de bits ; chaque bit a une place et un rôle.

Un nom de bit est une position

Un registre comme UCSR0A contient **8 bits**. Les noms définis dans avr/io.h désignent des positions. Un **masque** sélectionne le bit voulu : déplacer 1 de k positions donne **1 << k**.

7	6	5	4	3	2	1	0
0	0	0	1	0	0	0	0

Exemple générique : k = 4 → masque 00010000

Expression générique	Ce qu'elle signifie
`R & masque`	Tester les bits choisis : résultat nul si tous sont à 0.
`R |= masque`	Mettre les bits choisis à 1 en conservant les autres.
`R &= ~masque`	Mettre les bits choisis à 0 en conservant les autres.
`R = valeur`	Écrire une configuration complète du registre.
`masque1 | masque2`	Réunir deux sélections de bits.
`!expression`	Négation logique : donne 1 si l'expression vaut 0.

À ne pas appliquer aveuglément

Ces opérations supposent un registre ordinaire. Un registre matériel peut contenir des bits **en lecture seule** ou des indicateurs effacés en écrivant 1. Vérifie toujours R/W et la description du bit.

Lire une trame 8N1

La vitesse des bits et la fréquence d’envoi des caractères sont deux choses.

8 : données

8 bits forment l’octet à transmettre.

N : parité

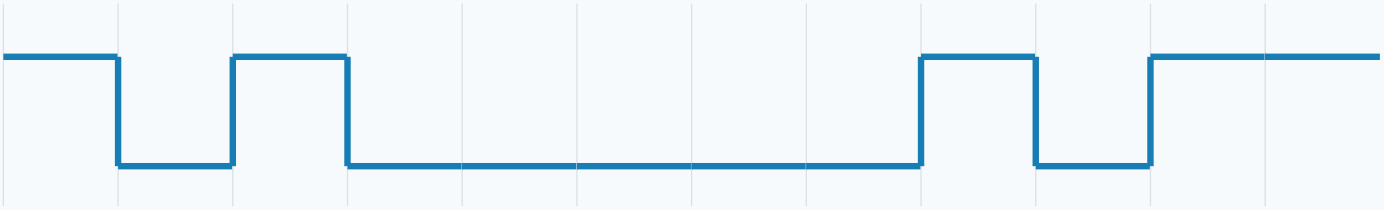
Aucun bit de parité ajouté.

1 : arrêt

Un bit de stop termine la trame.

Ligne au repos : **1**. Début de trame : **start = 0**. Les données arrivent avec **le bit de poids faible en premier** (d0, puis d1, etc.). Stop : **1**.

repos	start	d0	d1	d2	d3	d4	d5	d6	d7	stop	repos
1	0	1	0	0	0	0	0	1	0	1	1



Exemple : « A » = 65 = 0x41 = 01000001 (écriture d7…d0).

Sur le fil, les huit bits de « A » sont donc : **1, 0, 0, 0, 0, 0, 1, 0**. Le matériel ajoute start et stop ; tu lui fournis seulement l’octet.

Temps d’un bit

T_{bit} = 1 / débit

À 115200 bit/s :
environ 8,68 µs.

Temps d’une trame

8N1 = 1 + 8 + 1 = **10 bits**.

À 115200 bit/s :
environ 86,8 µs.

1 Hz veut dire un caractère chaque seconde. Cela ne change pas le débit : une courte trame est envoyée rapidement, puis la ligne reste au repos.

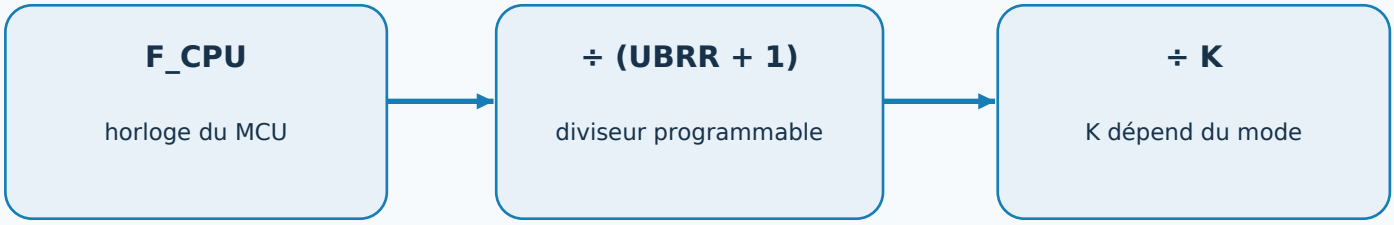
Sources : datasheet, §19.4 et figure 19-4 p.147-148

D’où vient le débit ?

F_CPU → diviseur UBRR → cadence des bits

F_CPU doit décrire la vraie fréquence

F_CPU est une constante de compilation exprimée en **hertz**. Elle doit correspondre à l’horloge effective du MCU, après division éventuelle. La définir ne change ni les fusibles, ni le quartz, ni le prédiviseur matériel.



Débit réel = F_CPU / [K × (UBRR + 1)]

Mode asynchrone	Valeur de K	Sélecteur
Normal	16	U2X0 = 0
Double vitesse	8	U2X0 = 1

Pourquoi le « + 1 » ?

Le compteur couvre les valeurs de UBRR jusqu’à 0 : cela fait **UBRR + 1** étapes. On inverse l’équation pour trouver le réglage :
UBRR idéal = F_CPU / (K × débit demandé) - 1.

Double vitesse : comprendre le choix

Avec le **même UBRR**, passer de K = 16 à K = 8 double le débit. Pour garder le débit demandé, il faut **recalculer UBRR**. Compare les erreurs obtenues ; le mode double vitesse n’est pas automatiquement le meilleur.

Sources : datasheet, §8.1 p.24 ; §8.11 p.32 ; §19.3.1-19.3.2 et tableau 19-1 p.145-146

UBRR : entier, puis deux morceaux

Le débit demandé est une cible ; le registre impose un diviseur entier.

Calcule la valeur idéale, examine les entiers voisins, puis recalcule le **débit réel** pour chacun. Compare la valeur absolue de l’erreur :

Erreur (%) = 100 × (débit réel / débit demandé - 1)

Un exemple différent de ton exercice

À **8 MHz et 9600 bit/s**, en mode normal : UBRR idéal ≈ 51,083. Avec UBRR = 51, le débit réel ≈ 9615,4 bit/s, soit environ **+0,16 %**. Ce calcul illustre la méthode, pas ton réglage final.

UBRR a 12 bits utiles, de 0 à 4095. Il est réparti sur deux registres de 8 bits : H contient les 4 bits de poids fort, L les 8 bits de poids faible.

UBRR0H

7	6	5	4	3	2	1	0
0	0	0	0	b11	b10	b9	b8

4 bits réservés à écrire à 0

UBRR0L

7	6	5	4	3	2	1	0
b7	b6	b5	b4	b3	b2	b1	b0

8 bits de poids faible

Découper une valeur : exemple fictif

Si une valeur calculée était **0x345** : partie haute = **0x03**, partie basse = **0x45**.
Un décalage à droite de 8 positions isole la partie haute. Un masque 0xFF ou une conversion vers un octet conserve la partie basse.

Calcul en C : les détails qui comptent

La division entre entiers **tronque**. Des constantes avec le suffixe **UL** évitent les produits en entier 16 bits sur AVR. Garde une expression liée à **F_CPU** et **UART_BAUDRATE** ; ne fige pas seulement une valeur numérique.

Sources : datasheet, §19.10.5 p.162 ; §19.11 et tableaux p.163-165

UCSR0A : lire le matériel

Un indicateur décrit un état ; il n’active pas l’émetteur.

7	6	5	4	3	2	1	0
RXC0	TXC0	UDRE0	FE0	DOR0	UPE0	U2X0	MPCM0

UDRE0 : tampon libre

1 : le tampon d’émission peut recevoir un octet.
0 : le tampon contient encore un octet à transférer.

TXC0 : émission achevée

Une trame entière a quitté le registre à décalage, et aucun nouvel octet n’attend dans le tampon.

Deux étages dans l’émetteur



Quand l’octet passe du tampon au registre à décalage, **UDRE0 peut déjà repasser à 1** alors que des bits sortent encore sur TXD. La place libre et la fin d’émission sont donc deux états différents.

U2X0 : un réglage dans un registre d’état

UCSR0A mélange des indicateurs et des réglages. **U2X0** sélectionne le facteur 8 ou 16 en mode asynchrone. RXC0 et les indicateurs FE0, DOR0, UPE0 concernent surtout la réception.

Attention à TXC0 : écrire 1 l’efface

TXC0 est un indicateur **W1C** : write one to clear. Une lecture-modification-écriture avec |= peut réécrire un 1 lu et effacer cet indicateur. Pour configurer UCSR0A, compose consciemment les bits écrits ; les bits d’état en lecture seule ne se « règlent » pas.

Sources : datasheet, §19.6.3 p.151-152 ; §19.10.2 p.159-160

UCSR0B : autoriser les fonctions

Émission, réception, interruptions et troisième bit de taille

7	6	5	4	3	2	1	0
RXCIE0	TXCIE0	UDRIE0	RXEN0	TXEN0	UCSZ02	RXB80	TXB80

Bit(s)	Rôle
TXEN0	Active l’émetteur. La fonction TXD prend le contrôle de PD1.
RXEN0	Active le récepteur. Utile si le MCU doit recevoir des données.
RXCIE0 / TXCIE0 / UDRIE0	Autorisent des interruptions sur les événements de l’USART.
UCSZ02	Troisième bit du champ qui choisit la taille des données.
RXB80 / TXB80	Neuvième bit de données pour les trames à 9 bits.

Scrutation (polling)

Le programme **lit lui-même** un indicateur et attend si nécessaire. Les interruptions UART ne sont pas nécessaires pour cette méthode.

Interruption

Le matériel peut appeler une routine quand un événement arrive. Il faut des autorisations et une routine adaptée. C’est une autre méthode.

À décider toi-même

Dans cet exercice, les données vont du **MCU vers le PC**. Identifie ce qui doit être activé. Choisis une méthode simple d’attente ; un en-tête autorisé n’oblige pas à utiliser toutes ses possibilités.

Sources : datasheet, §19.6 p.150 ; §19.10.3 p.160 ; §13.3.3 p.71

UCSR0C : décrire la trame

La taille des données se règle sur UCSR0B ET UCSR0C.

7	6	5	4	3	2	1	0
UMSEL01	UMSEL00	UPM01	UPM00	USBS0	UCSZ01	UCSZ00	UCPOL0

Champ	Ce qu’il choisit	Où vérifier
UMSEL01:0	Mode asynchrone, synchrone ou SPI	Tableau 19-4, p.161
UPM01:0	Absence de parité, parité paire ou impaire	Tableau 19-5, p.161
USBS0	Nombre de bits de stop	Tableau 19-6, p.161
UCSZ02:0	Nombre de bits de données	Tableau 19-7, p.162
UCPOL0	Polarité de l’horloge en mode synchrone	Description, p.162

Un champ réparti dans deux registres

B : UCSZ02 → **C : UCSZ01 puis UCSZ00**

Comment lire le tableau de taille

Lis toujours le triplet dans l’ordre **UCSZ02, UCSZ01, UCSZ00**. Exemple distinct : 0, 0, 1 correspond à 6 bits. À toi de retrouver la ligne pour 8 bits, puis de traduire chaque position dans le bon registre.

Sources : datasheet, §19.4 p.147-148 ; §19.10.4, tableaux 19-4 à 19-7 p.161-162

Fonctions et temps

Ce que chaque partie doit savoir faire, avant d’écrire son corps en C

Initialiser une fois

Une fonction **uart_init** prépare le débit, le mode, le format et les fonctions nécessaires. Ces réglages précèdent le premier envoi.

Choisis aussi le comportement des bits inutilisés.

Transmettre un caractère

```
void uart_tx(char c);
```

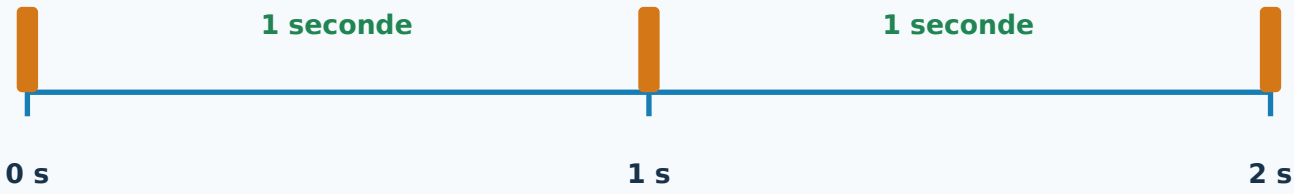
char c : un caractère en argument.
void : aucune valeur retournée.
; : déclaration de la fonction.

Exemple : 'A' est un caractère ; "A" est une chaîne.

Une déclaration n’est pas une définition

Le prototype annonce le nom, l’argument et le type de retour. La définition contient le corps entre accolades. **c** est un paramètre : il doit pouvoir porter n’importe quel caractère, pas uniquement celui demandé par l’exercice.

Fréquence f = 1 / période T. Pour 1 Hz, deux débuts d’envoi successifs sont séparés d’environ une seconde.



Chaque petite barre orange représente une trame très courte. **Le dessin n’est pas à l’échelle** : la durée de la trame est minuscule devant la seconde entre les envois.

Choisir une temporisation

util/delay.h fournit des attentes actives en millisecondes. F_CPU doit être défini avant son inclusion ; active les optimisations et utilise une durée connue à la compilation. Une attente en boucle donne une cadence approximative ; un timer permet de mieux contrôler la période.

En-têtes autorisés : avr/io.h pour les registres, util/delay.h pour l’attente, avr/interrupt.h pour les interruptions (facultatif ici).

Sources : datasheet, §19.5 p.148 ; §19.6.1 p.150 • AVR-LibC, util/delay.h (voir p.13)

Relier le MCU au PC

L’ordinateur doit voir un port série et utiliser les mêmes paramètres.

Liaison minimale pour recevoir sur le PC



TX de l’un va vers RX de l’autre.

Si ta carte a déjà une interface USB-série

Utilise son connecteur USB selon le schéma de la carte. Sinon, il faut une interface **USB-UART à niveaux logiques compatibles** avec le MCU. Une broche UART logique ne se branche pas directement sur les signaux USB ou sur un port RS-232.

Côté terminal : GNU Screen

Exemple Linux, en adaptant le nom du périphérique :

```
screen /dev/ttyUSB0 115200,cs8,-parenb,-cstopb
```

cs8 : 8 bits de données ; **-parenb** : parité désactivée ; **-cstopb** : un bit de stop. Le périphérique peut aussi s’appeler /dev/ttyACM0. Sur macOS, le chemin sera différent.

Ce que tu dois observer

L’envoi du seul caractère demandé produit une suite sur **la même ligne**. Sans retour chariot ni saut de ligne, le terminal n’invente pas de nouvelle ligne. Vérifie la cadence à l’œil et, au besoin, avec un instrument.

Sources : datasheet, §1.1 p.4 ; §13.3.3 p.70-71 • GNU Screen, Window Types (voir p.13)

À toi de construire les réglages

Complète ces cases avant d’écrire le programme : elles guident, sans corriger.

Décision	Ta réponse / tes calculs
Horloge réelle et F_CPU	_____
Débit demandé	_____
UBRR normal, débit réel, erreur	_____
UBRR double, débit réel, erreur	_____
Mode choisi et justification	_____
Valeurs de UBRR0H et UBRR0L	_____
UMSEL01:0 ; UPM01:0 ; USBS0	_____
UCSZ02 ; UCSZ01 ; UCSZ00	_____
Émetteur / récepteur / interruptions	_____
Indicateur à vérifier avant un dépôt	_____
Registre qui reçoit le caractère	_____
Période et méthode d’attente	_____

Quatre questions pour vérifier ta compréhension

1. Pourquoi ne suffit-il pas de modifier DDRD et PORTD ?
2. Pourquoi la valeur UBRR n’est-elle pas le débit 115200 ?
3. Comment un tampon peut-il être libre alors que TXD transmet encore ?
4. Pourquoi une erreur sur F_CPU peut-elle fausser à la fois débit et attente ?

Sources à ouvrir : p.146, p.159-162 et p.163-165 de la datasheet fournie

Déchiffrer la datasheet

Exemple réel : la petite table de UCSRnA ne décrit pas ton code final.

Le « n » veut dire numéro d'USART

La datasheet utilise des noms génériques : UCSRnA, UDRn, UBRRn. Sur l'ATmega328P, l'interface concernée est **USART0** : remplace n par 0. Cela donne UCSR0A, UDR0, UBRR0H / UBRR0L.

EXTRAIT ORIGINAL • PAGE 159

Bit	7	6	5	4	3	2	1	0	
	RXCn	TXCn	UDREn	FEn	DORn	UPEn	U2Xn	MPCMn	UCSRnA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

Ligne à lire	Interprétation
Bit : 7 … 0	Numéro de la position, du poids fort au poids faible.
Nom : RXCn, TXCn…	Fonction associée à la position ; n devient 0 ici.
Read/Write : R	Lecture seule : l'état est produit par le matériel.
Read/Write : R/W	Lecture et écriture, avec les règles de chaque bit.
Initial Value	État après reset ; ce n'est pas la configuration à copier.

La description du bit fait foi

R/W ne suffit pas : TXC0 s'efface en écrivant 1. De même, les bits réservés de UBRR0H doivent être écrits à 0. Lis **la table ET le texte**. Les pages 149 et 150 comportent du code d'exemple : ce guide n'en reproduit pas la solution.

Source : extrait de la datasheet fournie, §19.10.2 p.159 ; §19.10.5 p.162

Où retrouver chaque information

Les numéros ci-dessous correspondent à ta version, pas à toutes les datasheets.

Dans le PDF fourni, le **numéro affiché par le lecteur et le numéro imprimé sont identiques**. Cherche le chapitre **19. USART0**, pas « USART in SPI Mode » du chapitre 20.

Ce que tu cherches	Section / tableau	Page(s)
Broches TXD, RXD ; priorité sur GPIO	§13.3.3 ; tableau 13-9	70-71
Horloge et division système	§8.1 ; §8.11	24 ; 32
Vue d’ensemble de l’USART	§19.2 ; figure 19-1	143-144
Formules de débit ; mode double	§19.3.1-19.3.2 ; tab.19-1	145-146
Start, données, parité, stop	§19.4 ; figure 19-4	147-148
Rôle de l’initialisation	§19.5	148
Émission ; tampon libre / fin	§19.6.1 ; §19.6.3	150-152
UDR ; UCSR A et B	§19.10.1 à §19.10.3	159-160
Mode, parité, stop, taille	§19.10.4 ; tab.19-4 à 19-7	161-162
UBRR haut/bas ; bits réservés	§19.10.5	162
Débits réels et erreurs par horloge	§19.11 ; tab.19-9 à 19-12	163-165

Ce qui relève des outils, pas de la datasheet

AVR-LibC, util/delay.h : définition de F_CPU, optimisations, délais constants et attente active.
Ouvrir la documentation officielle AVR-LibC

GNU Screen, Window Types : port série, débit et options de format.
Ouvrir le manuel officiel GNU Screen

Schémas : redessinés pour expliquer les mécanismes. Présentation inspirée de ton guide GPIO ; rappels de C et exercices rédigés pour ce document.